



INSTITUTO TECNOLÓGICO Y DE ESTUDIOS

SUPERIORES DE MONTERREY

Actividad 5.2 Ejercicio de programación 2

**Adriana González Ugalde
A00194173**

TC4017 Pruebas de software y aseguramiento de la calidad

Dr. Gerardo Padilla Zárate

15 de febrero de 2026

Tabla de contenido

OBJETIVO Y AMBIENTE DE DESARROLLO	1
ESTRUCTURA DEL REPOSITORIO.....	1
PROGRAMA 1 – COMPUTE STATISTICS.....	2
Descripción del problema:	2
Requerimientos recibidos	2
Consideraciones y especificaciones adicionales sobre la implementación	2
Ejecución del programa	4
Resultados obtenidos	4
Verificación con pylint	6
PROGRAMA 2 – CONVERTER.....	ERROR! BOOKMARK NOT DEFINED.
Descripción del problema:	Error! Bookmark not defined.
Requerimientos recibidos	Error! Bookmark not defined.
Consideraciones y especificaciones adicionales sobre la implementación	Error! Bookmark not defined.
Ejecución del programa	Error! Bookmark not defined.
Resultados obtenidos	Error! Bookmark not defined.
Verificación con pylint	Error! Bookmark not defined.
PROGRAMA 3 – COUNT WORDS	ERROR! BOOKMARK NOT DEFINED.
Descripción del problema:	Error! Bookmark not defined.
Requerimientos recibidos	Error! Bookmark not defined.
Consideraciones y especificaciones adicionales sobre la implementación	Error! Bookmark not defined.
Ejecución del programa	Error! Bookmark not defined.
Resultados obtenidos	Error! Bookmark not defined.
Verificación con pylint	Error! Bookmark not defined.
CONCLUSIONES / REFLEXIONES	6
REFERENCIAS (FORMATO APA)	7

Objetivo y ambiente de desarrollo

El objetivo de este documento es presentar la implementación, ejecución y validación de un ejercicios de programación desarrollado en Python, aplicando criterios de calidad de software y utilizando las herramientas flake8 y pylint para hacer un análisis estático del código y verificar el cumplimiento del estándar PEP-8. Ambas herramientas detectan diferentes tipos de problemas: pylint evalúa estructura, diseño y calidad general, mientras que flake8 se enfoca principalmente en estilo y formato.

El orden en el que aplicaré las herramientas será flake8 primero y después pylint sobre el código ya corregido de los defectos que pueda marcar flake8. Esto porque primero buscaré corregir los errores de faltas al estándar PEP8 y después, sobre un código bien formatead, con estilo correcto, los de calidad y diseño de lógica.

Los programas fueron desarrollados utilizando Python y ejecutados desde la línea de comandos mediante la terminal integrada de Visual Studio Code.

Estructura del Repositorio

El **repositorio** se encuentra en Git, con el nombre **A00194173-Pruebas-y-Calidad** y tiene la estructura necesaria para contener este ejercicio y los posteriores de la materia, siguiendo la sugerencia del profesor Gerardo Padilla, lo cual me parece muy bueno para que sea más uniforme para encontrar las evidencias del ejercicio.

Este ejercicio en particular, *4.2. Ejercicio de programación 1*, se encuentra en la **carpeta A00194173_A4.2**, nombre solicitado en el ejercicio.

La estructura de directorios del repositorio para esta actividad es:

A00194173-Pruebas-y-Calidad	← Repositorio
└─ A00194173_A5.2	← Actividad 5.2
├─ Results	← Resultados pbas: .txt y screenshots
├─ Source	← Código fuente
└─ Test	← Datos de prueba
├─ TC1	← Datos para el caso de prueba 1
├─ TC2	← Datos para el caso de prueba 2
└─ TC3	← Datos para el caso de prueba 3

Programa – Compute sales

Descripción del problema:

El ejercicio consiste en desarrollar un programa en Python que lea dos archivos JSON, uno conteniendo un catálogo de productos con precios y otro con todas las ventas de la compañía, y calcule el total de todas esas ventas, manejando datos inválidos y reportando los resultados tanto en pantalla como en un archivo de salida.

Requerimientos recibidos

Los requerimientos implementados son los detallados en el documento de la asignación y que especifican:

Req1. The program shall be invoked from a command line. The program shall receive two files as parameters. The first file will contain information in a JSON format about a catalogue of prices of products. The second file will contain a record for all sales in a company.

Req 2. The program shall compute the total cost for all sales included in the second JSON archive. The results shall be print on a screen and on a file named SalesResults.txt. The total cost should include all items in the sale considering the cost for every item in the first file.

The output must be human readable, so make it easy to read for the user.

Req 3. The program shall include the mechanism to handle invalid data in the file. Errors should be displayed in the console and the execution must continue.

Req 4. The name of the program shall be compute_sales.py

Req 5. The minimum format to invoke the program shall be as follows:

python compute_sales.py priceCatalogue.json salesRecord.json

Req 6. The program shall manage files having from hundreds of items to thousands of items.

Req 7. The program should include at the end of the execution the time elapsed for the execution and calculus of the data. This number shall be included in the results file and on the screen.

Req 8. Be compliant with PEP8.

Consideraciones y especificaciones adicionales sobre la implementación

Adicional a los requerimientos de la asignación, se recibieron precisiones, y se tomaron decisiones sobre la implementación, para poder cumplir con los requerimientos y con las salidas esperadas.

A continuación se detalla cuáles fueron.

1. Nombre del archivo de código fuente

El requerimiento solicita que el archivo de código fuente tenga el nombre **computeSales.py**, cuya estructura **no es adecuada de acuerdo** a los estándares de **PEP8**, por lo que **se cambió a** notación snake case: **compute_sales.py**

2. Campo de relación entre los dos archivos de entrada

A falta de una indicación específica y revisando los archivos de prueba, se hará la relación entre los archivos de entrada por medio del campo de **title en el catálogo** y el campo **producto en el archivo de ventas**.

3. Tipos de errores verificados en el programa

Los tipos de errores a verificar en el código son los detectados en los archivos de datos de entrada recibidos, y algunos otros que suelen ser comunes en los archivos de tipo JSON.

Errores de archivo

- Archivo no encontrado
- No se puede abrir

Errores estructurales

- El contenido del archivo JSON no es lista
- Sus elementos no son objeto

Errores en catálogo

- Campo title faltante
- Campo title vacío
- Campo price inválido
- Campo price negativo
- Campo title duplicado

Errores en ventas

- Campo Product faltante
- Campo Product vacío
- Campo Quantity inválido
- Campo Quantity no entero
- Campo Producto no existe en catálogo

En las ventas, las cantidades negativas no se consideran inválidas, se consideran una devolución y como tales se restarán del total acumulado.

En estos casos el programa continúa, reporta como línea inválida pero no detiene la ejecución, sólo suma líneas válidas.

4. Estructura del archivo de salida

Los requerimientos no especifican el contenido ni la estructura de la salida, a excepción de que se debe calcular el total de ventas. La salida a pantalla y a archivo que se presentará es:

Total de ventas: (cantidad monetaria total acumulada de las ventas)

Líneas leídas: (líneas de ventas en el archivo)

Ventas válidas: (líneas de ventas con datos válidos)

Ventas inválidas: (líneas de ventas con algún error)

Devoluciones: (líneas de ventas con cantidad negativa)

Tiempo de ejecución (segundos):

Ejecución del programa

El programa se ejecutó desde la línea de comandos en la consola, como fue indicado, recibiendo como parámetro el archivo de prueba correspondiente (TC1.txt, TC2.txt... TC7.txt).

Ejemplo:

```
python compute_sales.py..\Test\TC1\ TC1.ProductList.json ..\Test\TC1\ TC1.sales.json
```

Resultados obtenidos

El nombre del archivo de salida es el solicitado en los requerimientos, **SalesResults**, y para facilitar la trazabilidad de los casos de prueba, **el nombre del archivo de salida incluye el identificador del archivo de ventas de cada caso de prueba**, generando un archivo de resultados independiente por cada caso de prueba.

Ejemplo:

Entrada: TC1.sales.json

Salida: **SalesResults TC1.txt**

En el **subfolder Screenshots del folder Results** se encuentran las **capturas de pantalla de los resultados de las pruebas, con nombres que incluyen Ejec** para indicar ejecución, del tipo 2 Ejec..., 4 Ejec... m 7 Ejec.... **Los numerales al inicio del nombre de archivo indican orden en la ejecución de Flake8, Pylint y de los diferentes casos de prueba. El orden numérico da una idea del proceso de construcción, modificaciones e iteraciones en el mismo.**

Los archivos de resultados y screenshots se encuentra en los siguientes directorios:

.

└─ Results	← Resultados pbas: .txt y screenshots
└─ SalesResultsTC1.txt	← Datos para el caso de prueba 1
└─ SalesResultsTC2.txt	← Datos para el caso de prueba 2
└─ SalesResultsTC3.txt	← Datos para el caso de prueba 3
└─ Screenshots	← Capturas de pantalla
└─ 1 flake8 P1-1.png	← Ejecución de Flake8
└─ 1 flake8 P1-2.png	← Ejecución de Flake8
└─ 1 flake8 P1-3.png	← Ejecución de Flake8
└─ 2 2 pylint P1-1.png	← Ejecución de Pylint
└─ 2 2 pylint P1-2.png	← Ejecución de Pylint
└─ 3 Ejec. TC1.png	← Ejecución del Programa para TC1
└─ 4 Ejec. TC2 - Error en total.png	← Ejecución del Programa para TC2
└─ 5 flake8 P1-1.png	← Ejecución de Flake8 tras correcciones
└─ 5 flake8 P1-1.png	← Ejecución de Flake8
└─ 6 pylint P1-1.png	← Ejecución de Pylint tras correcciones
└─ 7 flake8 P1-1.png	← Ejecución de Flake8 tras correcciones
└─ 7 flake8 P1-2.png	← Ejecución de Flake8 tras correcciones
└─ 8 pylint P1-1.png	← Ejecución de Pylint tras correcciones
└─ 9 Ejec. TC2 - Correcto.png	← Ejecución del Programa para TC2
└─ 10 Ejec. TC1 - Regression Test Correcto.png	← Ejec de Program para TC1
└─ 11 Ejec. TC3 - Correcto.png	← Ejecución del Programa para TC3
└─ Source	← Código fuente
└─ Test	← Datos de prueba
└─ TC1	← Datos para el caso de prueba 1
└─ TC2	← Datos para el caso de prueba 2
└─ TC3	← Datos para el caso de prueba 3

En el caso específico del caso de prueba TC2, el archivo de prueba contenía cantidades de compra negativas. Al calcular el total de ventas yo las estaba considerando como inválidas, pero al validar los resultados esperados en realidad se debían tratar como devoluciones, por lo que tuve que modificar la lógica del programa, volver a ejecutar revisiones con flake 8 y pylint, y hacer una prueba de regresión test al programa con los datos del TC1 para validar que todo estuviera correcto. Todo esto se puede ver en el orden de los screenshots de resultados.

Verificación con flake8 y pylint

El código fue analizado, antes de ser ejecutado, con las herramientas de análisis flake8 y pylint, en ese orden, como una estrategia escalonada de validación: eliminar primero los errores de formato y estilo del estándar PEP8, con lo que el código queda limpio en su estructura y eso facilita un análisis más profundo. Después ejecutar pylint para evaluar problemas de estructura del programa, complejidad de funciones, argumentos, demasiadas variables locales, nomenclatura, y buenas prácticas de programación.

Las herramientas se aplicaron hasta que ambas resultaron en 0 recomendaciones de correcciones corridas una después de la otra, como se muestra en el archivo de screenshot 8 pylint P1-1.png

Las recomendaciones reportadas por las herramientas fueron implementadas y documentadas mediante capturas de pantalla. Los nombres de los archivos llevan un numeral al inicio seguido por la palabra flake8 o pylint. Están intercalados con los ejercicios de ejecución para indicar en qué orden se ejecutó cada cosa:

- flake8 = se ejecutó flake8 sobre el código
- pylint = se ejecutó pylint sobre el código
- Ejec = se ejecutó el código con un archivo de prueba

Conclusiones / Reflexiones

Este ejercicio me permitió aplicar tanto revisiones manuales que generalmente aplico a mi código, y también usar las herramienta de análisis de código flake8 y pylint para revisar el cumplimiento de un estándar de implementación o guía de estilo como es PEP8, pero además, pensar en la estrategia de uso de las mismas para obtener el mejor provecho de ambas.

Creo que las diferentes herramientas de aseguramiento de la calidad deben ser evaluadas y utilizadas de tal manera que cada una de ellas no sólo aporte valor al resultado sino que fortalezca el uso de las herramientas subsecuentes, y con eso potenciar el objetivo para el cual se usa cada una.

En este caso usamos sólo dos herramientas, pero en un desarrollo completo de una aplicación, el orden en el que se utilicen además checklist, revisiones por pares, revisiones colegiadas, inspecciones, etcétera, permite que los defectos más superficiales se puedan eliminar con herramientas menos pesadas, de modo que cada iteración de prácticas de calidad repercuta en un producto final que cumpla con los estándares de calidad que se comprometen.

Referencias (FORMATO APA)

O'Regan, G. (2019). *Concise Guide to Software Testing* (1st. ed.). Springer Publishing Company, Incorporated.

Python Software Foundation. (s. f.). *PEP 8 — Style Guide for Python Code*. Recuperado el 11 de febrero de 2026, de <https://peps.python.org/pep-0008>

PyNative. (s. f.). *Python JSON programming*. Recuperado el 11 de febrero de 2026, de <https://pynative.com/python/json/>

Python Software Foundation. (s. f.). *Python Tutorial — Python 3 documentation*. Recuperado el 11 de febrero de 2026, de <https://docs.python.org/3/tutorial/index.html>

PyCQA. (s. f.). *Flake8 — Style Guide enforcement*. Recuperado el 11 de febrero de 2026, de <https://flake8.pycqa.org/en/latest/>

Luminousmen. (s. f.). *Python Static Analysis Tools*. Recuperado el 11 de febrero de 2026, de <https://luminousmen.com/post/python-static-analysis-tools>